Software Requirements Specification for
Task Manager
Version 4
Prepared by Daukeyeva Dilnaz
Technological University of Lodz
12.03.2026

Table of Contents

Section	Page
Revision History	2
1. Introduction	2
1.1. Purpose	2
1.2. Document Conventions	2
1.3. Intended Audience and Reading Suggestions	2
1.4. Product Scope	2
1.5. References	2
2. Overall Description	2
2.1. Problem Statement	2
2.2. Product Perspective	2
2.3. Product Features	3
2.4. User Classes and Characteristics	3
2.5. Operating Environment	3
2.6. Design and Implementation Constraints	3
2.7. User Documentation	3
2.8. Assumptions and Dependencies	3
3. System Features	3
3.1. User Authentication	3
3.2. Task Management	3
4. External Interface Requirements	3
4.1. User Interfaces Overview	3
4.2. Hardware Interfaces	3
4.3. Software Interfaces	4
5. System Features/Modules	4
5.1. Task Creation Module	4
5.1.1. Description and Priority	4
5.1.2. Stimulus/Response Sequences	4
5.1.3. Functional Requirements	4
6. Nonfunctional Requirements	4
6.1. Performance (NF-1.1)	4
6.2. Security (NF-2.1)	4
6.3. Reliability (NF-3.1)	4
7.1. Use Case Diagram	5
7.2. Class Diagram	6
7.3. Behavioral Diagram	7

Revision History

Name	Date	Reason for Changes	Version
Dilnaz	12.03.26	Initial version	1
Dilnaz	19.03.26	Use case diagram added	2
Dilnaz	26.03.26	Class diagram added	3
Dilnaz	09.04.26	Behavioral diagram added	4

1. Introduction

1.1. Purpose
The purpose of this document is to specify the software requirements for the Task Manager application, Version 1. This specification covers the core functionality of task creation, organization, and basic system performance attributes

1.2. Document Conventions
This document follows the IEEE 830 standard. All mandatory requirements are identified using the keyword "shall". Detailed requirements (Section 5) inherit the priority levels established in their parent system features (Section 3)

1.3. Intended Audience and Reading Suggestions
Developers: Focus on Section 5 (Functional Requirements) and Section 4 (Interfaces).
Project Managers: Review Section 1 and Section 2 for scope and constraints.
Testers: Focus on Sections 5 and 6 to plan how to test the software.
Reading Sequence: Start with the Introduction (Section 1) followed by the Overall Description (Section 2) before diving into technical details in Sections 4 through 6

1.4. Product Scope
The Task Manager is a tool to help users manage tasks. It keeps track of tasks and deadlines

2. Overall Description

2.1 Problem Statement
Currently, users managing multiple tasks often face "information overload," leading to missed deadlines and poor prioritization. Existing manual methods lack real-time synchronization and automated reminders. The Task Manager addresses these pains by providing a centralized, digital environment to ensure all professional and personal responsibilities are tracked and verified through a testable interface.

2.2. Product Perspective
This product is a new, self-contained web-based application. It is designed to operate independently and does not require integration with existing legacy systems for its initial release

2.3. Product Features
User Account Management.
Task Creation and Organization.
Priority and Deadline Settings

2.4. User Classes and Characteristics
Standard Users: People who want simple task tracking
Power Users: Professionals who use features

2.5. Operating Environment
The software works on web browsers and is compatible with both Windows and macOS operating systems

2.6. Design and Implementation Constraints
Language: The application shall be developed using Python-based frameworks.
Security: All data must be kept secure with encryption

2.7. User Documentation
Users will be provided with an integrated online help section and a PDF-based "Getting Started" guide

2.8. Assumptions and Dependencies
Assumptions: Users will have an internet connection
Dependencies: The system depends on email services for account verification

3. System Features

3.1. User Authentication
Users can securely access their task lists

3.2. Task Management
Enables the core functionality of creating, modifying, and viewing task items

4. External Interface Requirements

4.1. User Interfaces Overview
The system will feature a Graphical User Interface (GUI) where users interact via buttons, forms, and navigation menus. Feedback for actions (like saving a task) will be displayed via success or error banners at the top of the screen

4.2. Hardware Interfaces
The software does not require specific hardware beyond standard computer peripherals (keyboard, mouse) and a network interface for internet access

4.3. Software Interfaces

The application will interface with a relational database for data storage and utilize standard libraries for password hashing and email transmission

5. System Features/Modules

5.1 Task Creation Module

5.1.1 Description and Priority

Users can enter tasks. This is a high-priority feature

5.1.2 Stimulus/Response Sequences

Stimulus: User triggers Complete Task
Response: The system shall prompt for confirmation, update the status to 'Completed ' and move the item to the Archive view
Stimulus: User submits the task form with valid data.
Response: The system saves the task and updates the dashboard

5.1.3 Functional Requirements

FR-01: Users can create an account with an email
FR-02: Upon clicking "Save," the system shall validate that the task title is not empty.
FR-03: Users can assign a date to each task.
FR-04: The system shall allow users to categorize tasks by priority level (High, Medium, Low).
FR-05: The system shall allow users to mark a task as "Completed" to remove it from the active dashboard view
FR-06: The system shall allow Power Users to sort tasks by multiple criteria simultaneously
FR-07: The system shall allow users to create custom Categories and link them to multiple tasks

6. Nonfunctional Requirements

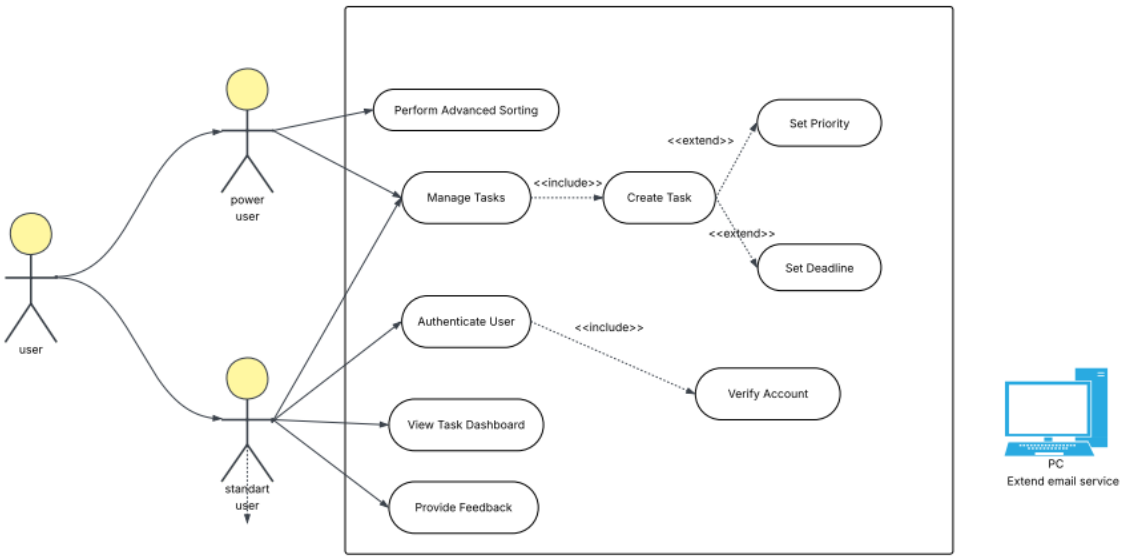
6.1. Performance

NF-1.1: The system shall respond to user queries in less than 2 seconds
NF-2.1: All user passwords shall be hashed using the Argon2 algorithm
NF-3.1: The system shall maintain 99.5% uptime during standard business hours

6.2. Security

NF-2.1: All user passwords shall be hashed using a strong hashing algorithm
NF-2.2: The system shall automatically log out after 30 minutes of inactivity

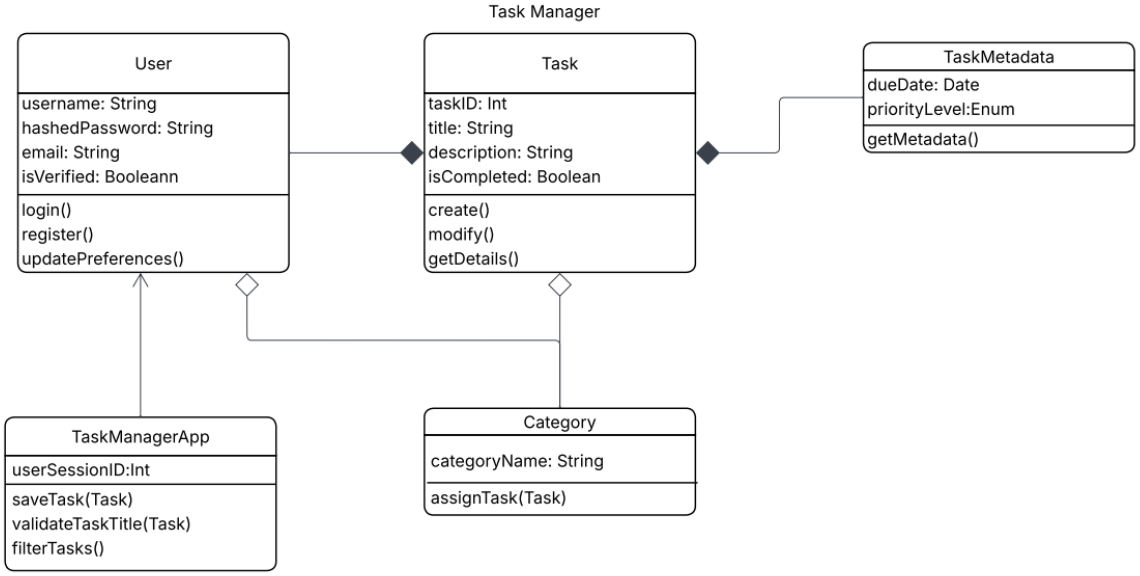
7.1. The Use Case diagram for Task Manager:



This UML Use Case Diagram illustrates the functional scope of the Task Manager system and the interactions between its various actors. The diagram establishes a clear system boundary, defining the application as a standalone web-based tool.

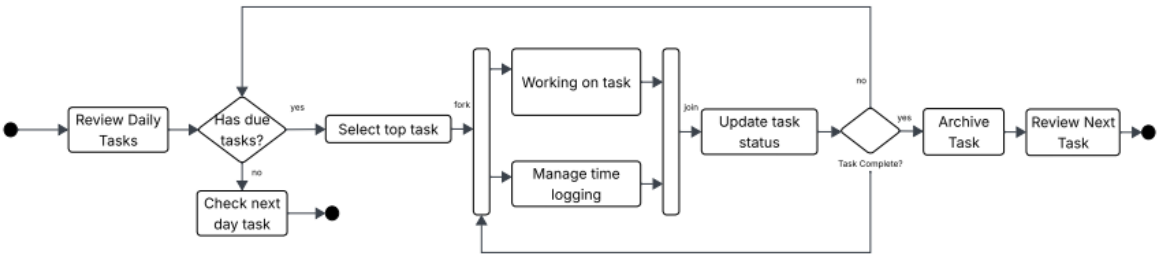
Primary Actors: The system serves two types of human users: the Standard User, who handles basic task tracking and dashboard viewing, and the Power User, a professional who utilizes advanced organizational features.
Secondary Actor: The External Email Service is a separate system integrated specifically for verifying user accounts during the authentication process.
Core Relationships:
Authentication: Accessing private data requires a user to "Authenticate," which includes a mandatory account verification step.
Task Management: The "Manage Tasks" function always includes the ability to "Create Tasks".
Optional Features: When creating a task, users have the choice to "Set Priorities" or "Set Deadlines" to better monitor their work.
Advanced Features: The "Power User" is the only actor with the privilege to "Perform Advanced Sorting"

7.2. The UML Class diagram for Task Manager:



The UML class diagram defines the structural architecture of the system, centered around the Task entity and its relationship to the User. The system is managed by the TaskManagerApp controller.
The User class manages account security and profiles (login, registration), maintaining a strong composition relationship with Task; this ensures tasks are strictly tied to a specific user account. A Category class allows for logical grouping through an aggregation relationship, enabling users to organize tasks without strict ownership constraints

7.3. The Behavioral Diagram of Task Manager:



A task starts as Pending and moves to Assigned once it's yours, becoming In Progress while you work, though you can put it On Hold if you need a break. This lifecycle is driven by your daily routine, which starts by Reviewing Tasks and picking the top priority, then using a Fork to work and Log Time at the same time. These actions meet at a Join to Update Status, where you decide if the job is done; if not, you loop back to keep working, but if it is, you Archive it and move to the next task. This simple loop keeps the system updated until every task is finished and you’ve checked what's due for tomorrow